Serial Brun — starting a bench soak from the front panel

Ian Ameline · version 1.58 · MIT licence

`Serial_brun.tspa` puts the bench engine behind a settings screen on the instrument, so a lap can be started with nobody connected. It is the same `brun.soak()` the host drives — a second door into one engine, not a second engine. Everything `tools/run_bench.py` did still works, and a host-driven load leaves this screen available for the next lap.

It is bench scaffolding and is not the Serial Decode app. It carries the whole app inside it, because the engine measures through `sdec.acquire()`, but it installs separately and shows its own screen. Do not install it on an instrument you want as a serial analyser.

What it cannot do, and why that shapes the screen

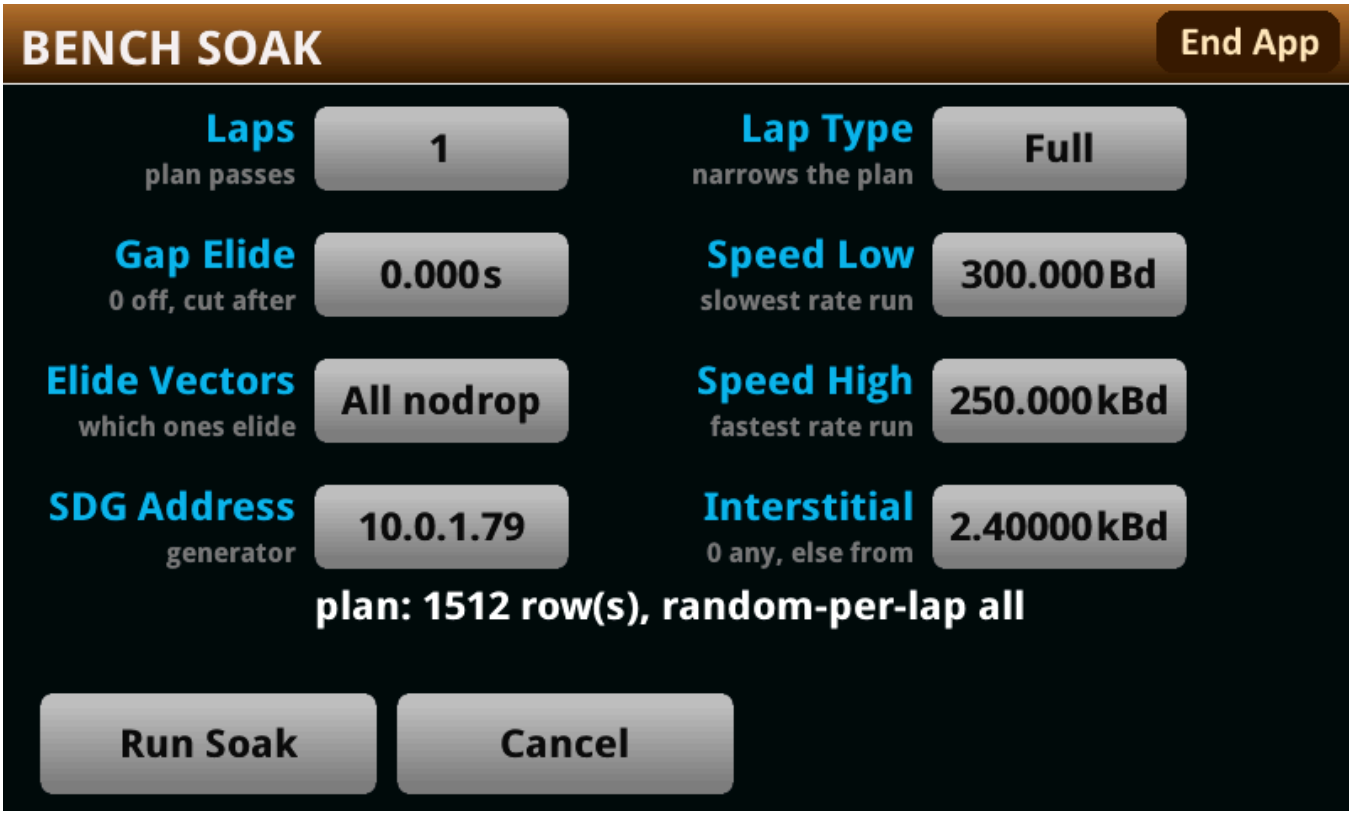
It cannot draw a plan. `tools/soakplan.py` draws the cell list on the host and the USB key carries it as `/usb1/SERDEC/PLAN.CSV`. Re-deriving that draw on the instrument would be a second implementation of the one thing every lap is compared against — the amplitude, the offset and the wait are all keyed off the cell index, so a plan drawn twice is two different experiments wearing one name.

So the screen **narrows a plan that is already there**. Push one once:

```
python3 tools/run_bench.py --push-plan                          # the full draw
python3 tools/run_bench.py --push-plan --spec v98d:std,v99a:std # a targeted subset
```

and from then on laps start at the panel. The screen reports what it found, and refuses to start without it. A `--spec` push **overwrites** `PLAN.CSV`, so the row count on the screen is the only thing that says which plan is actually on the key.

The screen



The plan line is read off the key when the screen is built, so it describes what is actually there rather than what was last pushed from this host. The count is the plan's own `# rows=` header — data rows, which is six fewer than the line count `run_bench.py --push-plan` reports, that figure including the header and comment lines.

field	what it does
Laps	passes over the plan, 1 to 210. There is deliberately no "forever": an unbounded run started at the panel has nobody watching a host log that would say so.
Gap Elide	the detector window in seconds, `0` for off. Off is the default — elision is the feature under test, not the normal way to run a lap.
Elide Vectors	which vectors elide when Gap Elide is on. `v98d v99d v99a` are the only three whose gaps clear two windows at both 9600 and 19200 Bd.
SDG Address	the generator, as a pick list. There is no string-entry object in this firmware's set, and a list cannot produce the typo that costs a lap to discover. The live value leads it.
Lap Type	narrows the plan — see below.
Speed Low / **Speed High**	the rate window in Bd, inclusive at both ends. Defaults span the ladder: 300 is the slowest rate drawn, 250 000 the decoder's own ceiling.
Interstitial	a floor under the *drawn* rates only, `0` for none. Standard rates still play below it. At the bottom of the ladder a non-standard rate costs the most seconds and tests the least.

Lap Type

	what plays
Full	every row the plan holds, which is what the host-driven path runs

	what plays
Normal	standard rates only — no drawn interstitial or edge rate
Gap targeted	only vectors that declare an idle run
Adversarial	only vectors known to produce errors

`Adversarial` is derived, not listed: it is `barb.expect`'s `loud` entries — the vectors the plan drives into conditions where refusing is the documented right answer — plus every vector carrying a gap, whose seams cost bytes until the decoder splice lands. A vector added to either table is covered here without anybody remembering to update a third list.

`Gap targeted` is keyed off `brun.elide_gapbits` for the same reason.

Running one

Press **Run Soak**. The settings screen is torn down and the engine's own status screen takes over, which is the same screen a host-driven lap shows: lap and cell, the waveform playing, the measured rate, the error line and a spinner that keeps moving through the parts of a capture that take seconds.

Two refusals can appear on the plan line instead, and both are worth reading rather than retrying:

`REFUSED: no plan on the key` — nothing to run. Push one.

`REFUSED: a smoke has run on this power cycle` — a soak started after a smoke fails *every* cell with `acq: line is idle (no transitions)`, and only a power cycle clears it. This has happened three times on this bench; the last was 705 cells of 705. The host guard that catches it lives in `run_bench.py`, so the panel carries its own.

Stopping is the front-panel **TRIGGER** key. The run ends after the current waveform and closes the record, which is what commits it on a FAT key. Cutting the power costs at most the last row.

Reading the result

The record is `/usb1/SERDEC/SOAK.csv` on the key and the panel is not the oracle — the error line is a health indicator, and a lap's real verdict comes from the host judge:

```
python3 tools/run_bench.py --fetch --no-load
python3 tools/judge_bench.py --verbose out/bench/SOAK.csv
```

The error line reads `Errors: N  Bad: b/t  nodec N  flag N  wrong N/M`. `Bad:` is bytes and `wrong:` is cells, and the difference matters: `wrong 5/132` says five cells are bad, where `Bad: 251/1313` says four fifths of the bytes were fine and the damage is localised. When the line will not fit the screen the per-class breakout is dropped and the count and the byte figure are kept.

Installing

```
python3 tools/package_tspa.py --bench          # writes Serial_brun.tspa
```

Copy it to the key's root and install it from **Menu > Scripts**. It needs the same USB key the record and the plan live on.

What it does not change

The host path is untouched. `run_bench.py --start --detach` behaves exactly as before, including its own smoke-poison guard and the key pruning it does first. The panel's settings are applied to the same `brun.*` fields the host sets, and every filter defaults to nil — so a host-driven lap reads the rows it always did, whatever was last chosen on the glass.